实验四：SLR(1) 分析表生成

姓名：李祥熙　学号：2234412799

2026 年 5 月 30 日

目录

1 实验目的

本实验的目标如下：

- 目标一：**构建给定文法的 LR(0) 项目集规范族与状态转移图**
- 目标二：**计算文法符号的 FIRST/FOLLOW 集，为 SLR(1) 分析表提供依据**
- 目标三：**生成 SLR(1) 的 ACTION/GOTO 表并检测冲突**

2 实验过程

2.1 实验环境

- 操作系统：**Linux**
- 编程语言：**C**
- 编译器：**GCC**
- 其他工具：**终端与文本编辑器**

2.2 输入文法与预处理

本实验的输入为文本形式的文法规则（支持“|”表示同一产生式的多个候选式）。本次运行使用表达式文法（文件：`grammar_expr.txt`）：

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

预处理步骤为：

- 读取所有规则，识别非终结符与终结符集合。
- 自动增广文法，添加新的开始符号 E'。
- 将“|”分支拆分为多条产生式，便于后续计算。

2.3 实现步骤

1. **文法解析与增广**：读取输入、建立符号表并生成增广产生式。
2. **LR(0) 项目集**：计算闭包与 GOTO，构建规范族及状态转移图。
3. **FIRST/FOLLOW**：迭代求解 FIRST 与 FOLLOW 集（处理空产生式）。
4. **SLR(1) 表构造**：依据项目集与 FOLLOW 集填充 ACTION/GOTO 表，并检测冲突。

3 实验内容

3.1 任务一：LR(0) 项目集规范族构造

实现要点：

- 项目用 “(产生式编号, 圆点位置)” 表示，便于比较与排序。
- 通过闭包函数将 “A -> $\alpha.B\beta$”
- GOTO 基于某个符号移动圆点，再对结果求闭包。

代码片段（闭包与 GOTO）：

```
static void
closure (ItemSet *set, const Production *prods, const int *
   nonterm_first,
                const int *nonterm_count, const int *symbol_is_nonterm
                   )
{
    int changed = 1;
    while (changed)
        {
            changed = 0;
            for (int i = 0; i < set->count; ++i)
                {
                    Item it = set->items[i];
                    const Production *p = &prods[it.prod];
                    if (it.dot >= p->rhs_len)
                        {
                            continue;
                        }
                    int sym = p->rhs[it.dot];
                    if (!symbol_is_nonterm[sym])
                        {
                            continue;
                        }
                    int nt_index = sym;
                    int start = nonterm_first[nt_index];
                    int cnt = nonterm_count[nt_index];
                    for (int k = 0; k < cnt; ++k)
                        {
                            Item new_item = { start + k, 0 };
                            if (!itemset_contains (set, &new_item))
```

```
                                {
                                    itemset_add (set, &new_item);
                                    changed = 1;
                                }
                            }
                        }
                    }
}
```

说明：该片段展示了闭包算法的核心逻辑：当圆点后为非终结符时，将其所有产生式加入项目集并反复迭代，直到集合稳定。

3.2 任务二：SLR(1) ACTION/GOTO 表生成

实现要点：

- 对每个状态的项目：
  - 若圆点前进符号为终结符，则填写 ACTION 的移进（shift）。
  - 若为归约项目，则对该产生式左部的 FOLLOW 集填写归约（reduce）。
  - 若为增广产生式的归约项目，则在 “” *accept*
- 对每个非终结符的状态转移填写 GOTO。
- 若同一格出现不同动作，判为 SLR(1) 冲突。

4 实验结果

4.1 运行与输出

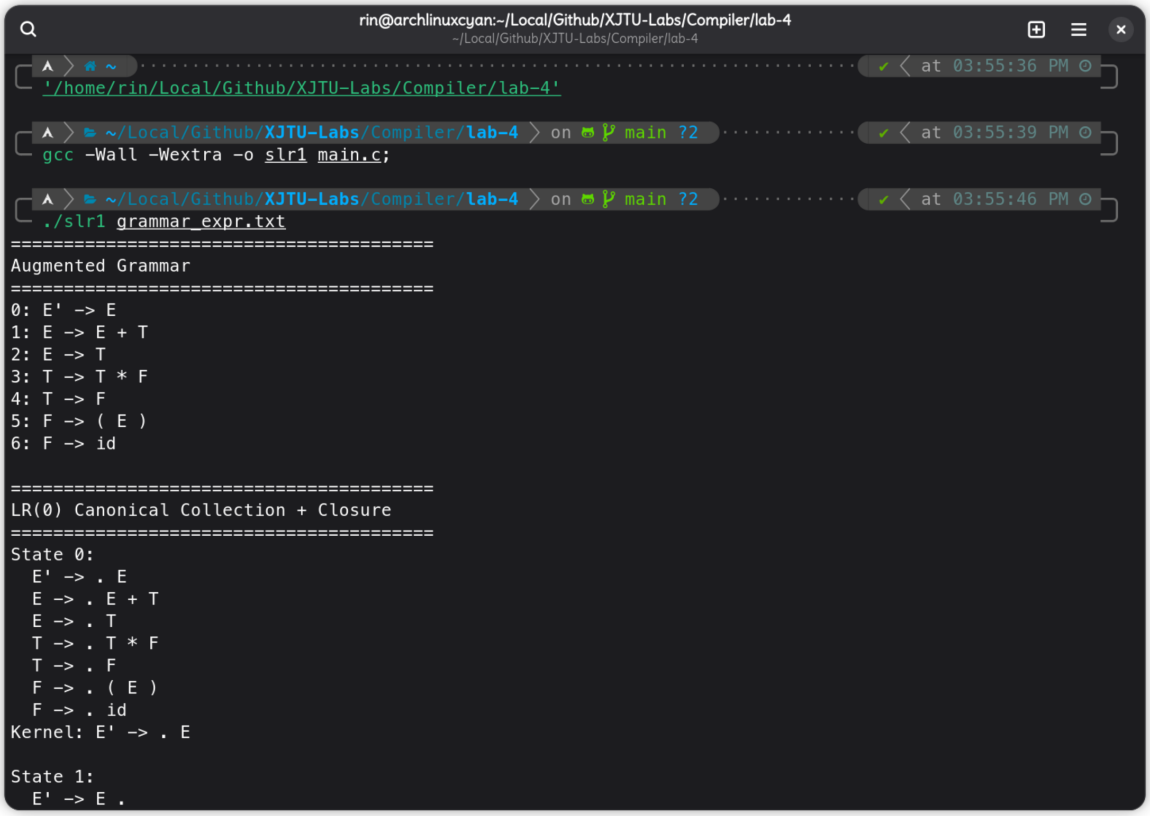

图 1: 编译与运行示例

说明：图 1 展示了在实验目录下使用 GCC 编译，并执行 “`./slr1 grammar_expr.txt E`” 的过程，便于验证程序能够顺利完成分析表生成。

4.2 输出片段与分析

程序输出包含：增广文法、LR(0) 项目集、FOLLOW 集、ACTION 表与 GOTO 表。以下是程序输出：

```
========================================
Augmented Grammar
========================================
0: E' -> E
1: E -> E + T
2: E -> T
3: T -> T * F
4: T -> F
```

```
5: F -> ( E )
6: F -> id

========================================
LR(0) Canonical Collection + Closure
========================================
State 0:
  E' -> . E
  E -> . E + T
  E -> . T
  T -> . T * F
  T -> . F
  F -> . ( E )
  F -> . id
Kernel: E' -> . E

State 1:
  E' -> E .
  E -> E . + T
Kernel: E' -> E ., E -> E . + T

State 2:
  E -> T .
  T -> T . * F
Kernel: E -> T ., T -> T . * F

State 3:
  T -> F .
Kernel: T -> F .

State 4:
  E -> . E + T
  E -> . T
  T -> . T * F
  T -> . F
  F -> . ( E )
  F -> ( . E )
  F -> . id
```

```
Kernel: F -> ( . E )

State 5:
  F -> id .
Kernel: F -> id .

State 6:
  E -> E + . T
  T -> . T * F
  T -> . F
  F -> . ( E )
  F -> . id
Kernel: E -> E + . T

State 7:
  T -> T * . F
  F -> . ( E )
  F -> . id
Kernel: T -> T * . F

State 8:
  E -> E . + T
  F -> ( E . )
Kernel: E -> E . + T, F -> ( E . )

State 9:
  E -> E + T .
  T -> T . * F
Kernel: E -> E + T ., T -> T . * F

State 10:
  T -> T * F .
Kernel: T -> T * F .

State 11:
  F -> ( E ) .
Kernel: F -> ( E ) .
```

```
========================================
State Transition Graph
========================================
0 --E--> 1
0 --T--> 2
0 --F--> 3
0 --(--> 4
0 --id--> 5
1 --+--> 6
2 --*--> 7
4 --E--> 8
4 --T--> 2
4 --F--> 3
4 --(--> 4
4 --id--> 5
6 --T--> 9
6 --F--> 3
6 --(--> 4
6 --id--> 5
7 --F--> 10
7 --(--> 4
7 --id--> 5
8 --+--> 6
8 --)--> 11
9 --*--> 7
========================================
FOLLOW Sets
========================================
FOLLOW(E) = { +, ), $ }
FOLLOW(T) = { +, *, ), $ }
FOLLOW(F) = { +, *, ), $ }
FOLLOW(E') = { $ }

========================================
ACTION Table
========================================
State   +       *       (       )       id      $
0                       s4              s5
```

```
1       s6                                      acc
2       r2      s7              r2              r2
3       r4      r4              r4              r4
4                       s4              s5
5       r6      r6              r6              r6
6                       s4              s5
7                       s4              s5
8       s6                      s11
9       r1      s7              r1              r1
10      r3      r3              r3              r3
11      r5      r5              r5              r5

========================================
GOTO Table
========================================
State   E       T       F
0       1       2       3
1
2
3
4       8       2       3
5
6               9       3
7                       10
8
9
10
11

No SLR(1) conflicts detected.
```

说明：该输出片段来自本次运行的完整输出文件（`output.txt`）。其中 FOLLOW 集用于归约项填表，ACTION 表显示各状态对终结符的移进/归约/接受动作。最后的 "No SLR(1) conflicts detected" 说明该文法在 SLR(1) 下无冲突。

5 总结

本实验成功完成了 SLR(1) 分析表的构造：

- 基于 LR(0) 项目集规范族完成了状态转移图的构造。

- 计算得到 FIRST/FOLLOW 集并用于 SLR(1) 表填充。
- 最终生成 ACTION/GOTO 表，且对表达式文法未检测到冲突。

后续可扩展：增加对更多文法示例的测试，或进一步支持 LALR(1) 合并状态的分析表生成。

附录：实验源代码

A.1 主要源码：main.c

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE 1024
#define MAX_LINES 512
#define MAX_PRODS 512
#define MAX_RHS 32
#define MAX_SYMBOLS 256
#define MAX_STATES 256
#define MAX_ITEMS 4096
#define MAX_NAME 64

typedef struct
{
    int lhs;
    int rhs[MAX_RHS];
    int rhs_len;
} Production;

typedef struct
{
    int prod;
    int dot;
} Item;

typedef struct
{
    Item items[MAX_ITEMS];
    int count;
```

```
} ItemSet;

typedef enum
{
    ACT_NONE = 0,
    ACT_SHIFT,
    ACT_REDUCE,
    ACT_ACCEPT
} ActionType;

typedef struct
{
    ActionType type;
    int value;
} ActionCell;

static void
trim (char *s)
{
    size_t n = strlen (s);
    while (n > 0 && (s[n - 1] == '\n' || s[n - 1] == '\r'))
        {
            s[n - 1] = '\0';
            --n;
        }
    size_t i = 0;
    while (s[i] && isspace ((unsigned char)s[i]))
        {
            ++i;
        }
    if (i > 0)
        {
            memmove (s, s + i, strlen (s + i) + 1);
        }
    n = strlen (s);
    while (n > 0 && isspace ((unsigned char)s[n - 1]))
        {
            s[n - 1] = '\0';
            --n;
        }
}
```

```
static int
is_blank (const char *s)
{
    while (*s)
        {
            if (!isspace ((unsigned char)*s))
                {
                    return 0;
                }
            ++s;
        }
    return 1;
}

static int
starts_with (const char *s, const char *prefix)
{
    return strncmp (s, prefix, strlen (prefix)) == 0;
}

static int
find_arrow (const char *s)
{
    const char *p = strstr (s, "->");
    if (!p)
        {
            return -1;
        }
    return (int)(p - s);
}

static int
name_index (char names[][MAX_NAME], int count, const char *name)
{
    for (int i = 0; i < count; ++i)
        {
            if (strcmp (names[i], name) == 0)
                {
                    return i;
                }
```

```
        }
    return -1;
}

static int
add_name (char names[][MAX_NAME], int *count, const char *name)
{
    int idx = name_index (names, *count, name);
    if (idx >= 0)
        {
            return idx;
        }
    if (*count >= MAX_SYMBOLS)
        {
            return -1;
        }
    strncpy (names[*count], name, MAX_NAME - 1);
    names[*count][MAX_NAME - 1] = '\0';
    (*count)++;
    return (*count) - 1;
}

static int
is_nonterminal (char nonterms[][MAX_NAME], int nonterm_count, const
   char *s)
{
    return name_index (nonterms, nonterm_count, s) >= 0;
}

static int
item_equal (const Item *a, const Item *b)
{
    return a->prod == b->prod && a->dot == b->dot;
}

static int
itemset_contains (const ItemSet *set, const Item *it)
{
    for (int i = 0; i < set->count; ++i)
        {
            if (item_equal (&set->items[i], it))
```

```
                {
                    return 1;
                }
        }
    return 0;
}

static void
itemset_add (ItemSet *set, const Item *it)
{
    if (!itemset_contains (set, it))
        {
            if (set->count < MAX_ITEMS)
                {
                    set->items[set->count++] = *it;
                }
        }
}

static int
item_cmp (const void *a, const void *b)
{
    const Item *ia = (const Item *)a;
    const Item *ib = (const Item *)b;
    if (ia->prod != ib->prod)
        {
            return ia->prod - ib->prod;
        }
    return ia->dot - ib->dot;
}

static void
itemset_sort (ItemSet *set)
{
    qsort (set->items, (size_t)set->count, sizeof (Item), item_cmp);
}

static int
itemset_equal (const ItemSet *a, const ItemSet *b)
{
    if (a->count != b->count)
```

```
        {
            return 0;
        }
    for (int i = 0; i < a->count; ++i)
        {
            if (!item_equal (&a->items[i], &b->items[i]))
                {
                    return 0;
                }
        }
    return 1;
}

static void
closure (ItemSet *set, const Production *prods, const int *
   nonterm_first,
                const int *nonterm_count, const int *symbol_is_nonterm
                   )
{
    int changed = 1;
    while (changed)
        {
            changed = 0;
            for (int i = 0; i < set->count; ++i)
                {
                    Item it = set->items[i];
                    const Production *p = &prods[it.prod];
                    if (it.dot >= p->rhs_len)
                        {
                            continue;
                        }
                    int sym = p->rhs[it.dot];
                    if (!symbol_is_nonterm[sym])
                        {
                            continue;
                        }
                    int nt_index = sym;
                    int start = nonterm_first[nt_index];
                    int cnt = nonterm_count[nt_index];
                    if (start < 0 || cnt <= 0)
                        {
```

```
                            continue;
                        }
                    for (int k = 0; k < cnt; ++k)
                        {
                            Item new_item;
                            new_item.prod = start + k;
                            new_item.dot = 0;
                            if (!itemset_contains (set, &new_item))
                                {
                                    itemset_add (set, &new_item);
                                    changed = 1;
                                }
                        }
                }
        }
}

static void
goto_set (const ItemSet *src, int symbol, ItemSet *dst,
                    const Production *prods, int prod_count, const int
                       *nonterm_first,
                    const int *nonterm_count, int nonterm_total,
                    const int *symbol_is_nonterm)
{
    dst->count = 0;
    for (int i = 0; i < src->count; ++i)
        {
            Item it = src->items[i];
            const Production *p = &prods[it.prod];
            if (it.dot < p->rhs_len && p->rhs[it.dot] == symbol)
                {
                    Item moved;
                    moved.prod = it.prod;
                    moved.dot = it.dot + 1;
                    itemset_add (dst, &moved);
                }
        }
    (void)prod_count;
    (void)nonterm_total;
    closure (dst, prods, nonterm_first, nonterm_count,
       symbol_is_nonterm);
```

```
    itemset_sort (dst);
}

static void
print_production (const Production *p, char symbols[][MAX_NAME])
{
    printf ("%s ->", symbols[p->lhs]);
    if (p->rhs_len == 0)
        {
            printf (" epsilon");
        }
    else
        {
            for (int i = 0; i < p->rhs_len; ++i)
                {
                    printf (" %s", symbols[p->rhs[i]]);
                }
        }
}

static void
print_item (const Item *it, const Production *prods, char symbols[][
   MAX_NAME])
{
    const Production *p = &prods[it->prod];
    printf ("%s ->", symbols[p->lhs]);
    for (int i = 0; i < p->rhs_len; ++i)
        {
            if (i == it->dot)
                {
                    printf (" .");
                }
            printf (" %s", symbols[p->rhs[i]]);
        }
    if (it->dot == p->rhs_len)
        {
            printf (" .");
        }
}

static void
```

```
collect_kernel (const ItemSet *set, ItemSet *kernel)
{
    kernel->count = 0;
    for (int i = 0; i < set->count; ++i)
        {
            Item it = set->items[i];
            if (it.dot > 0 || it.prod == 0)
                {
                    itemset_add (kernel, &it);
                }
        }
    itemset_sort (kernel);
}

static int
parse_grammar_lines (char lines[][MAX_LINE], int line_count,
                                        char nonterms[][MAX_NAME], int
                                            *nonterm_count,
                                        char symbols[][MAX_NAME], int
                                           *symbol_count,
                                        int *start_symbol, Production
                                           *prods, int *prod_count)
{
    *nonterm_count = 0;
    *symbol_count = 0;
    *prod_count = 0;

    for (int i = 0; i < line_count; ++i)
        {
            char buf[MAX_LINE];
            strncpy (buf, lines[i], sizeof (buf) - 1);
            buf[sizeof (buf) - 1] = '\0';
            trim (buf);
            if (is_blank (buf) || starts_with (buf, "#") || starts_with
                (buf, "//"))
                {
                    continue;
                }
            int arrow = find_arrow (buf);
            if (arrow < 0)
                {
```

```
                return 0;
            }
        buf[arrow] = '\0';
        buf[arrow + 1] = '\0';
        char lhs[MAX_NAME];
        strncpy (lhs, buf, sizeof (lhs) - 1);
        lhs[sizeof (lhs) - 1] = '\0';
        trim (lhs);
        if (lhs[0] == '\0')
            {
                return 0;
            }
        add_name (nonterms, nonterm_count, lhs);
    }

if (*nonterm_count == 0)
    {
        return 0;
    }

for (int i = 0; i < *nonterm_count; ++i)
    {
        add_name (symbols, symbol_count, nonterms[i]);
    }

*start_symbol = 0;

for (int i = 0; i < line_count; ++i)
    {
        char buf[MAX_LINE];
        strncpy (buf, lines[i], sizeof (buf) - 1);
        buf[sizeof (buf) - 1] = '\0';
        trim (buf);
        if (is_blank (buf) || starts_with (buf, "#") || starts_with
            (buf, "//"))
            {
                continue;
            }

        int arrow = find_arrow (buf);
        if (arrow < 0)
```

```
        {
            return 0;
        }

    buf[arrow] = '\0';
    char rhs_part[MAX_LINE];
    strncpy (rhs_part, buf + arrow + 2, sizeof (rhs_part) - 1);
    rhs_part[sizeof (rhs_part) - 1] = '\0';

    char lhs[MAX_NAME];
    strncpy (lhs, buf, sizeof (lhs) - 1);
    lhs[sizeof (lhs) - 1] = '\0';
    trim (lhs);
    if (lhs[0] == '\0')
        {
            return 0;
        }

    int lhs_sym = name_index (symbols, *symbol_count, lhs);
    if (lhs_sym < 0)
        {
            return 0;
        }

    char *alt = rhs_part;
    while (alt)
        {
            char *bar = strchr (alt, '|');
            if (bar)
                {
                    *bar = '\0';
                }
            char alt_buf[MAX_LINE];
            strncpy (alt_buf, alt, sizeof (alt_buf) - 1);
            alt_buf[sizeof (alt_buf) - 1] = '\0';
            trim (alt_buf);

            Production p;
            p.lhs = lhs_sym;
            p.rhs_len = 0;
```

```
                if (alt_buf[0] == '\0' || strcmp (alt_buf, "epsilon
                   ") == 0
                        || strcmp (alt_buf, "eps") == 0)
                    {
                        p.rhs_len = 0;
                    }
                else
                    {
                        char *tok = strtok (alt_buf, " \t");
                        while (tok)
                            {
                                int sym;
                                if (is_nonterminal (nonterms, *
                                   nonterm_count, tok))
                                    {
                                        sym = name_index (symbols,
                                           *symbol_count, tok);
                                    }
                                else
                                    {
                                        sym = name_index (symbols,
                                           *symbol_count, tok);
                                        if (sym < 0)
                                            {
                                                sym = add_name (
                                                   symbols,
                                                   symbol_count,
                                                   tok);
                                            }
                                    }
                                if (sym < 0)
                                    {
                                        return 0;
                                    }
                                if (p.rhs_len >= MAX_RHS)
                                    {
                                        return 0;
                                    }
                                p.rhs[p.rhs_len++] = sym;
                                tok = strtok (NULL, " \t");
                            }
```

```
                        }

                    if (*prod_count >= MAX_PRODS)
                        {
                            return 0;
                        }
                    prods[(*prod_count)++] = p;

                    if (bar)
                        {
                            alt = bar + 1;
                        }
                    else
                        {
                            alt = NULL;
                        }
                }
        }

    return 1;
}

static int
add_to_set (unsigned char *set, int row, int col)
{
    unsigned char *cell = &set[row * MAX_SYMBOLS + col];
    if (*cell)
        {
            return 0;
        }
    *cell = 1;
    return 1;
}

static void
compute_first (const Production *prods, int prod_count, int
   symbol_count,
                               const int *symbol_is_nonterm, unsigned
                                  char *first,
                               unsigned char *first_eps)
{
```

```
    for (int i = 0; i < symbol_count; ++i)
        {
            first_eps[i] = 0;
            if (!symbol_is_nonterm[i])
                {
                    add_to_set (first, i, i);
                }
        }

    int changed = 1;
    while (changed)
        {
            changed = 0;
            for (int i = 0; i < prod_count; ++i)
                {
                    const Production *p = &prods[i];
                    if (p->rhs_len == 0)
                        {
                            if (!first_eps[p->lhs])
                                {
                                    first_eps[p->lhs] = 1;
                                    changed = 1;
                                }
                            continue;
                        }

                    int all_eps = 1;
                    for (int j = 0; j < p->rhs_len; ++j)
                        {
                            int sym = p->rhs[j];
                            for (int t = 0; t < symbol_count; ++t)
                                {
                                    if (first[sym * MAX_SYMBOLS + t])
                                        {
                                            if (add_to_set (first, p->
                                               lhs, t))
                                                {
                                                    changed = 1;
                                                }
                                        }
                                }
```

```
                            if (!first_eps[sym])
                                {
                                    all_eps = 0;
                                    break;
                                }
                        }
                    if (all_eps && !first_eps[p->lhs])
                        {
                            first_eps[p->lhs] = 1;
                            changed = 1;
                        }
                }
        }
}

static void
compute_follow (const Production *prods, int prod_count, int
   symbol_count,
                                const int *symbol_is_nonterm, const
                                   unsigned char *first,
                                const unsigned char *first_eps,
                                   unsigned char *follow,
                                unsigned char *follow_end, int
                                   start_symbol)
{
    memset (follow_end, 0, MAX_SYMBOLS * sizeof (unsigned char));
    follow_end[start_symbol] = 1;

    int changed = 1;
    while (changed)
        {
            changed = 0;
            for (int i = 0; i < prod_count; ++i)
                {
                    const Production *p = &prods[i];
                    for (int j = 0; j < p->rhs_len; ++j)
                        {
                            int sym = p->rhs[j];
                            if (!symbol_is_nonterm[sym])
                                {
                                    continue;
```

```
                }

            int beta_has_eps = 1;
            for (int k = j + 1; k < p->rhs_len; ++k)
                {
                    int next = p->rhs[k];
                    for (int t = 0; t < symbol_count;
                       ++t)
                        {
                            if (first[next *
                               MAX_SYMBOLS + t])
                                {
                                    if (add_to_set (
                                       follow, sym, t))
                                        {
                                            changed =
                                               1;
                                        }
                                }
                        }
                    if (!first_eps[next])
                        {
                            beta_has_eps = 0;
                            break;
                        }
                }

            if (j + 1 >= p->rhs_len)
                {
                    beta_has_eps = 1;
                }

            if (beta_has_eps)
                {
                    for (int t = 0; t < symbol_count;
                       ++t)
                        {
                            if (follow[p->lhs *
                               MAX_SYMBOLS + t])
                                {
                                    if (add_to_set (
```

```
                                                    follow, sym, t))
                                                     {
                                                          changed =
                                                             1;
                                                     }
                                                }
                                            }
                                        if (follow_end[p->lhs] && !
                                           follow_end[sym])
                                            {
                                                follow_end[sym] = 1;
                                                changed = 1;
                                            }
                                    }
                                }
                            }
                    }
}

static const char *
action_type_name (ActionType type)
{
    switch (type)
        {
        case ACT_SHIFT:
            return "shift";
        case ACT_REDUCE:
            return "reduce";
        case ACT_ACCEPT:
            return "accept";
        default:
            return "none";
        }
}

static int
set_action (ActionCell *cell, ActionType type, int value, int state,
                        const char *sym)
{
    if (cell->type != ACT_NONE && (cell->type != type || cell->value !=
        value))
```

```
        {
            printf ("Conflict at state %d, symbol %s: %s vs %s\n",
               state, sym,
                           action_type_name (cell->type),
                              action_type_name (type));
            return 1;
        }
    cell->type = type;
    cell->value = value;
    return 0;
}

static void
format_action (const ActionCell *cell, char *buf, size_t size)
{
    if (cell->type == ACT_SHIFT)
        {
            snprintf (buf, size, "s%d", cell->value);
        }
    else if (cell->type == ACT_REDUCE)
        {
            snprintf (buf, size, "r%d", cell->value);
        }
    else if (cell->type == ACT_ACCEPT)
        {
            snprintf (buf, size, "acc");
        }
    else
        {
            buf[0] = '\0';
        }
}

static void
print_follow_sets (int symbol_count, const int *symbol_is_nonterm,
                                     char symbols[][MAX_NAME], const
                                        unsigned char *follow,
                                     const unsigned char *follow_end)
{
    printf ("========================================\n");
    printf ("FOLLOW Sets\n");
```

```
    printf ("========================================\n");
    for (int i = 0; i < symbol_count; ++i)
        {
            if (!symbol_is_nonterm[i])
                {
                    continue;
                }
            printf ("FOLLOW(%s) = { ", symbols[i]);
            int first_item = 1;
            for (int t = 0; t < symbol_count; ++t)
                {
                    if (follow[i * MAX_SYMBOLS + t])
                        {
                            if (!first_item)
                                {
                                    printf (", ");
                                }
                            printf ("%s", symbols[t]);
                            first_item = 0;
                        }
                }
            if (follow_end[i])
                {
                    if (!first_item)
                        {
                            printf (", ");
                        }
                    printf ("$");
                }
            printf (" }\n");
        }
    printf ("\n");
}

static void
print_action_table (int state_count, const int *terminals, int
   term_count,
                                        char symbols[][MAX_NAME], const
                                            ActionCell *action_cells)
{
    printf ("========================================\n");
```

```
    printf ("ACTION Table\n");
    printf ("=======================================\n");
    printf ("%-8s", "State");
    for (int i = 0; i < term_count; ++i)
        {
            printf ("%-8s", symbols[terminals[i]]);
        }
    printf ("%-8s\n", "$");

    for (int s = 0; s < state_count; ++s)
        {
            printf ("%-8d", s);
            for (int i = 0; i < term_count + 1; ++i)
                {
                    const ActionCell *cell = &action_cells[s * (
                       MAX_SYMBOLS + 1) + i];
                    char buf[16];
                    format_action (cell, buf, sizeof (buf));
                    printf ("%-8s", buf);
                }
            printf ("\n");
        }
    printf ("\n");
}

static void
print_goto_table (int state_count, const int *nonterms, int
   nonterm_count,
                                    char symbols[][MAX_NAME], const int
                                        *goto_table)
{
    printf ("=======================================\n");
    printf ("GOTO Table\n");
    printf ("=======================================\n");
    printf ("%-8s", "State");
    for (int i = 0; i < nonterm_count; ++i)
        {
            printf ("%-8s", symbols[nonterms[i]]);
        }
    printf ("\n");
```

```
    for (int s = 0; s < state_count; ++s)
        {
            printf ("%-8d", s);
            for (int i = 0; i < nonterm_count; ++i)
                {
                    int val = goto_table[s * MAX_SYMBOLS + i];
                    if (val >= 0)
                        {
                            printf ("%-8d", val);
                        }
                    else
                        {
                            printf ("%-8s", "");
                        }
                }
            printf ("\n");
        }
    printf ("\n");
}

int
main (int argc, char **argv)
{
    char (*lines)[MAX_LINE] = malloc (sizeof (*lines) * MAX_LINES);
    if (!lines)
        {
            fprintf (stderr, "Out of memory.\n");
            return 1;
        }
    int line_count = 0;

    FILE *fp = stdin;
    if (argc >= 2)
        {
            fp = fopen (argv[1], "r");
            if (!fp)
                {
                    fprintf (stderr, "Cannot open file: %s\n", argv[1])
                        ;
                    return 1;
                }
```

```
    }

while (line_count < MAX_LINES && fgets (lines[line_count], MAX_LINE
   , fp))
    {
        trim (lines[line_count]);
        if (!is_blank (lines[line_count]))
            {
                line_count++;
            }
    }

if (fp != stdin)
    {
        fclose (fp);
    }

if (line_count == 0)
    {
        fprintf (stderr, "No grammar rules provided.\n");
        return 1;
    }

char (*nonterms)[MAX_NAME] = malloc (sizeof (*nonterms) *
   MAX_SYMBOLS);
char (*symbols)[MAX_NAME] = malloc (sizeof (*symbols) * MAX_SYMBOLS
   );
if (!nonterms || !symbols)
    {
        fprintf (stderr, "Out of memory.\n");
        return 1;
    }
int nonterm_count = 0;
int symbol_count = 0;
int start_symbol = 0;
Production *prods = malloc (sizeof (*prods) * MAX_PRODS);
if (!prods)
    {
        fprintf (stderr, "Out of memory.\n");
        return 1;
    }
```

```
    int prod_count = 0;

    if (!parse_grammar_lines (lines, line_count, nonterms, &
       nonterm_count,
                                                          symbols, &
                                                             symbol_count
                                                             , &
                                                             start_symbol
                                                             , prods,
                                                          &prod_count))
        {
            fprintf (stderr,
                             "Failed to parse grammar. Ensure lines
                                like: E -> E + T | T\n");
            return 1;
        }

    if (argc >= 3)
        {
            int idx = name_index (symbols, symbol_count, argv[2]);
            if (idx < 0)
                {
                    fprintf (stderr, "Start symbol '%s' not found in
                       grammar.\n",
                                     argv[2]);
                    return 1;
                }
            start_symbol = idx;
        }

    char aug_name[MAX_NAME];
    snprintf (aug_name, sizeof (aug_name), "%s'", symbols[start_symbol
       ]);
    if (name_index (symbols, symbol_count, aug_name) >= 0)
        {
            snprintf (aug_name, sizeof (aug_name), "%s0", symbols[
               start_symbol]);
        }

    int aug_sym = add_name (symbols, &symbol_count, aug_name);
    if (aug_sym < 0)
```

```
        {
            fprintf (stderr, "Too many symbols.\n");
            return 1;
        }

    Production aug;
    aug.lhs = aug_sym;
    aug.rhs_len = 1;
    aug.rhs[0] = start_symbol;

    Production all_prods[MAX_PRODS];
    int all_prod_count = 0;
    all_prods[all_prod_count++] = aug;
    for (int i = 0; i < prod_count; ++i)
        {
            all_prods[all_prod_count++] = prods[i];
        }

    int *symbol_is_nonterm = calloc (MAX_SYMBOLS, sizeof (int));
    if (!symbol_is_nonterm)
        {
            fprintf (stderr, "Out of memory.\n");
            return 1;
        }
    for (int i = 0; i < nonterm_count; ++i)
        {
            int idx = name_index (symbols, symbol_count, nonterms[i]);
            if (idx >= 0)
                {
                    symbol_is_nonterm[idx] = 1;
                }
        }
    symbol_is_nonterm[aug_sym] = 1;

    int *nonterm_first = malloc (sizeof (int) * MAX_SYMBOLS);
    int *nonterm_prod_count = malloc (sizeof (int) * MAX_SYMBOLS);
    if (!nonterm_first || !nonterm_prod_count)
        {
            fprintf (stderr, "Out of memory.\n");
            return 1;
        }
```

```
    for (int i = 0; i < MAX_SYMBOLS; ++i)
        {
            nonterm_first[i] = -1;
            nonterm_prod_count[i] = 0;
        }

    for (int i = 0; i < all_prod_count; ++i)
        {
            int lhs = all_prods[i].lhs;
            if (nonterm_first[lhs] < 0)
                {
                    nonterm_first[lhs] = i;
                }
            nonterm_prod_count[lhs]++;
        }

    for (int i = 0; i < symbol_count; ++i)
        {
            if (symbol_is_nonterm[i] && nonterm_prod_count[i] == 0)
                {
                    fprintf (stderr, "Nonterminal '%s' has no
                       productions.\n",
                                    symbols[i]);
                    return 1;
                }
        }

    printf ("========================================\n");
    printf ("Augmented Grammar\n");
    printf ("========================================\n");
    for (int i = 0; i < all_prod_count; ++i)
        {
            printf ("%d: ", i);
            print_production (&all_prods[i], symbols);
            printf ("\n");
        }

    ItemSet *states = calloc (MAX_STATES, sizeof (ItemSet));
    if (!states)
        {
            fprintf (stderr, "Out of memory.\n");
```

```
            return 1;
        }
    int state_count = 0;
    int (*transitions)[MAX_SYMBOLS]
            = malloc (sizeof (*transitions) * MAX_STATES);
    if (!transitions)
        {
            fprintf (stderr, "Out of memory.\n");
            return 1;
        }
    for (int i = 0; i < MAX_STATES; ++i)
        {
            for (int j = 0; j < MAX_SYMBOLS; ++j)
                {
                    transitions[i][j] = -1;
                }
        }

    ItemSet start_set;
    start_set.count = 0;
    Item start_item;
    start_item.prod = 0;
    start_item.dot = 0;
    itemset_add (&start_set, &start_item);

    closure (&start_set, all_prods, nonterm_first, nonterm_prod_count,
                     symbol_is_nonterm);
    itemset_sort (&start_set);
    states[state_count++] = start_set;

    for (int i = 0; i < state_count; ++i)
        {
            for (int sym = 0; sym < symbol_count; ++sym)
                {
                    ItemSet next;
                    goto_set (&states[i], sym, &next, all_prods,
                       all_prod_count,
                                        nonterm_first,
                                           nonterm_prod_count,
                                           symbol_count,
                                        symbol_is_nonterm);
```

```
                    if (next.count == 0)
                        {
                            continue;
                        }
                    int existing = -1;
                    for (int k = 0; k < state_count; ++k)
                        {
                            if (itemset_equal (&states[k], &next))
                                {
                                    existing = k;
                                    break;
                                }
                        }
                    if (existing < 0)
                        {
                            if (state_count >= MAX_STATES)
                                {
                                    fprintf (stderr, "Too many states.\
                                       n");
                                    return 1;
                                }
                            states[state_count] = next;
                            existing = state_count;
                            state_count++;
                        }
                    transitions[i][sym] = existing;
                }
        }

    printf ("\n========================================\n");
    printf ("LR(0) Canonical Collection + Closure\n");
    printf ("========================================\n");

    for (int i = 0; i < state_count; ++i)
        {
            printf ("State %d:\n", i);
            for (int j = 0; j < states[i].count; ++j)
                {
                    printf ("  ");
                    print_item (&states[i].items[j], all_prods, symbols
                       );
```

```
                printf ("\n");
            }
        ItemSet kernel;
        collect_kernel (&states[i], &kernel);
        printf ("Kernel: ");
        for (int k = 0; k < kernel.count; ++k)
            {
                if (k > 0)
                    {
                        printf (", ");
                    }
                print_item (&kernel.items[k], all_prods, symbols);
            }
        printf ("\n\n");
    }

printf ("========================================\n");
printf ("State Transition Graph\n");
printf ("========================================\n");
for (int i = 0; i < state_count; ++i)
    {
        for (int sym = 0; sym < symbol_count; ++sym)
            {
                int to = transitions[i][sym];
                if (to >= 0)
                    {
                        printf ("%d --%s--> %d\n", i, symbols[sym],
                            to);
                    }
            }
    }

unsigned char *first
        = calloc (MAX_SYMBOLS * MAX_SYMBOLS, sizeof (unsigned char)
           );
unsigned char *follow
        = calloc (MAX_SYMBOLS * MAX_SYMBOLS, sizeof (unsigned char)
           );
unsigned char *first_eps = calloc (MAX_SYMBOLS, sizeof (unsigned
   char));
unsigned char *follow_end = calloc (MAX_SYMBOLS, sizeof (unsigned
```

```
    char));
if (!first || !follow || !first_eps || !follow_end)
    {
        fprintf (stderr, "Out of memory.\n");
        return 1;
    }

compute_first (all_prods, all_prod_count, symbol_count,
    symbol_is_nonterm,
                               first, first_eps);
compute_follow (all_prods, all_prod_count, symbol_count,
    symbol_is_nonterm,
                                  first, first_eps, follow,
                                     follow_end, aug_sym);
print_follow_sets (symbol_count, symbol_is_nonterm, symbols, follow
    ,
                                       follow_end);

int terminals[MAX_SYMBOLS];
int term_count = 0;
int term_col[MAX_SYMBOLS];
for (int i = 0; i < MAX_SYMBOLS; ++i)
    {
        term_col[i] = -1;
    }
for (int i = 0; i < symbol_count; ++i)
    {
        if (!symbol_is_nonterm[i])
            {
                term_col[i] = term_count;
                terminals[term_count++] = i;
            }
    }

int nonterms_out[MAX_SYMBOLS];
int nonterm_out_count = 0;
for (int i = 0; i < symbol_count; ++i)
    {
        if (symbol_is_nonterm[i] && i != aug_sym)
            {
                nonterms_out[nonterm_out_count++] = i;
```

```
                }
        }

    ActionCell *action
            = calloc (MAX_STATES * (MAX_SYMBOLS + 1), sizeof (
               ActionCell));
    int *goto_table = malloc (sizeof (int) * MAX_STATES * MAX_SYMBOLS);
    if (!action || !goto_table)
        {
            fprintf (stderr, "Out of memory.\n");
            return 1;
        }
    for (int i = 0; i < MAX_STATES * MAX_SYMBOLS; ++i)
        {
            goto_table[i] = -1;
        }

    int conflict = 0;
    for (int i = 0; i < state_count; ++i)
        {
            for (int j = 0; j < states[i].count; ++j)
                {
                    Item it = states[i].items[j];
                    const Production *p = &all_prods[it.prod];
                    if (it.dot < p->rhs_len)
                        {
                            int sym = p->rhs[it.dot];
                            if (!symbol_is_nonterm[sym])
                                {
                                    int to = transitions[i][sym];
                                    int col = term_col[sym];
                                    if (to >= 0 && col >= 0)
                                        {
                                            ActionCell *cell = &action[
                                               i * (MAX_SYMBOLS + 1) +
                                               col];
                                            if (set_action (cell,
                                               ACT_SHIFT, to, i,
                                               symbols[sym]))
                                                {
                                                    conflict = 1;
```

```
                                        }
                                    }
                                }
                            }
                        else
                            {
                                if (it.prod == 0)
                                    {
                                        ActionCell *cell
                                                = &action[i * (MAX_SYMBOLS
                                                   + 1) + term_count];
                                        if (set_action (cell, ACT_ACCEPT,
                                           0, i, "$"))
                                            {
                                                conflict = 1;
                                            }
                                    }
                                else
                                    {
                                        int lhs = p->lhs;
                                        for (int t = 0; t < symbol_count;
                                           ++t)
                                            {
                                                if (follow[lhs *
                                                   MAX_SYMBOLS + t])
                                                    {
                                                        int col = term_col[
                                                           t];
                                                        if (col >= 0)
                                                            {
                                                                ActionCell
                                                                   *cell
                                                                        = &
                                                                    action
                                                                    [
                                                                    i

                                                                    *

                                                                    (
                                                                    MAX_SYMBOLS
```

```
                                                    +
                                                    1)
                                                    +
                                                    col
                                                    ];
                                        if (
                                            set_action
                                             (cell,
                                            ACT_REDUCE
                                            , it.
                                            prod, i,
                                                 {
                                                      conflict
                                                      =
                                                      1;
                                                 }
                                        }
                                    }
                                }
                            if (follow_end[lhs])
                                {
                                    ActionCell *cell
                                            = &action[i * (
                                               MAX_SYMBOLS + 1)
                                                + term_count];
                                    if (set_action (cell,
                                       ACT_REDUCE, it.prod, i,
```

```
                                                    "$"))
                                                     {
                                                         conflict = 1;
                                                     }
                                                }
                                            }
                                        }
                                    }

                                for (int j = 0; j < nonterm_out_count; ++j)
                                    {
                                        int sym = nonterms_out[j];
                                        int to = transitions[i][sym];
                                        if (to >= 0)
                                            {
                                                goto_table[i * MAX_SYMBOLS + j] = to;
                                            }
                                    }
                            }

                        print_action_table (state_count, terminals, term_count, symbols,
                            action);
                        print_goto_table (state_count, nonterms_out, nonterm_out_count,
                            symbols,
                                                        goto_table);

                        if (conflict)
                            {
                                printf ("Grammar has SLR(1) conflicts.\n");
                            }
                        else
                            {
                                printf ("No SLR(1) conflicts detected.\n");
                            }

                        return 0;
                    }
```

A.2 运行命令

```
# 编译
gcc -Wall -Wextra -o slr1 main.c

# 运行 (表达式文法)
./slr1 grammar_expr.txt E
```